JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ReentrantReadWriteLock.WriteLock

java.lang.Object
  java.util.concurrent.locks.ReentrantReadWriteLock.WriteLock

All Implemented Interfaces:

Serializable, Lock

Enclosing class:

ReentrantReadWriteLock

```
public static class ReentrantReadWriteLock.WriteLock
extends Object
implements Lock, Serializable
```

The lock returned by method ReentrantReadWriteLock.writeLock().

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	WriteLock (ReentrantReadWriteLock	Constructor for use by subclasses.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	getHoldCount()	Queries the number of holds on this write lock by the current thread.
boolean	isHeldByCurrentThread()	Queries if this write lock is held by the current thread.
void	lock()	Acquires the write lock.

void	**lockInterruptibly**()	Acquires the write lock unless the current thread is interrupted.
Condition	**newCondition**()	Returns a Condition instance for use with this Lock instance.
String	**toString**()	Returns a string identifying this lock, as well as its lock state.
boolean	**tryLock**()	Acquires the write lock only if it is not held by another thread at the time of invocation.
boolean	**tryLock**(long timeout, TimeUnit unit)	Acquires the write lock if it is not held by another thread within the given waiting time and the current thread has not been interrupted.
void	**unlock**()	Attempts to release this lock.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

WriteLock

protected WriteLock(ReentrantReadWriteLock lock)

Constructor for use by subclasses.

Parameters:

lock - the outer lock object

Throws:

NullPointerException - if the lock is null

Method Details

lock

```
public void lock()
```

Acquires the write lock.

Acquires the write lock if neither the read nor write lock are held by another thread and returns immediately, setting the write lock hold count to one.

If the current thread already holds the write lock then the hold count is incremented by one and the method returns immediately.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until the write lock has been acquired, at which time the write lock hold count is set to one.

Specified by:

`lock` in interface `Lock`

lockInterruptibly

```
public void lockInterruptibly()
                        throws InterruptedException
```

Acquires the write lock unless the current thread is interrupted.

Acquires the write lock if neither the read nor write lock are held by another thread and returns immediately, setting the write lock hold count to one.

If the current thread already holds this lock then the hold count is incremented by one and the method returns immediately.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of two things happens:

- The write lock is acquired by the current thread; or
- Some other thread interrupts the current thread.

If the write lock is acquired by the current thread then the lock hold count is set to one.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the write lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock.

Specified by:

`lockInterruptibly` in interface `Lock`

Throws:

`InterruptedException` - if the current thread is interrupted

tryLock

```
public boolean tryLock()
```

Acquires the write lock only if it is not held by another thread at the time of invocation.

Acquires the write lock if neither the read nor write lock are held by another thread and returns immediately with the value `true`, setting the write lock hold count to one. Even when this lock has been set to use a fair ordering policy, a call to `tryLock()` *will* immediately acquire the lock if it is available, whether or not other threads are currently waiting for the write lock. This "barging" behavior can be useful in certain circumstances, even though it breaks fairness. If you want to honor the fairness setting for this lock, then use `tryLock(0, TimeUnit.SECONDS)` which is almost equivalent (it also detects interruption).

If the current thread already holds this lock then the hold count is incremented by one and the method returns `true`.

If the lock is held by another thread then this method will return immediately with the value `false`.

Specified by:

`tryLock` in interface `Lock`

Returns:

`true` if the lock was free and was acquired by the current thread, or the write lock was already held by the current thread; and `false` otherwise.

tryLock

```
public boolean tryLock(long timeout,
                       TimeUnit unit)
                throws InterruptedException
```

Acquires the write lock if it is not held by another thread within the given waiting time and the current thread has not been interrupted.

Acquires the write lock if neither the read nor write lock are held by another thread and returns immediately with the value `true`, setting the write lock hold count to one. If this lock has been set to use a fair ordering policy then an available lock *will not* be acquired if any other threads are waiting for the write lock. This is in contrast to the `tryLock()` method. If you want a timed `tryLock` that does permit barging on a fair lock then combine the timed and un-timed forms together:

```
if (lock.tryLock() ||
    lock.tryLock(timeout, unit)) {
  ...
}
```

If the current thread already holds this lock then the hold count is incremented by one and the method returns `true`.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of three things happens:

- The write lock is acquired by the current thread; or
- Some other thread interrupts the current thread; or
- The specified waiting time elapses

If the write lock is acquired then the value `true` is returned and the write lock hold count is set to one.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the write lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

If the specified waiting time elapses then the value `false` is returned. If the time is less than or equal to zero, the method will not wait at all.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock, and over reporting the elapse of the waiting time.

Specified by:

`tryLock` in interface `Lock`

Parameters:

`timeout` - the time to wait for the write lock

`unit` - the time unit of the timeout argument

Returns:

`true` if the lock was free and was acquired by the current thread, or the write lock was already held by the current thread; and `false` if the waiting time elapsed before the lock could be acquired.

Throws:

`InterruptedException` - if the current thread is interrupted

`NullPointerException` - if the time unit is null

unlock

```
public void unlock()
```

Attempts to release this lock.

If the current thread is the holder of this lock then the hold count is decremented. If the hold count is now zero then the lock is released. If the current thread is not the holder of this lock then `IllegalMonitorStateException` is thrown.

Specified by:

`unlock` in interface `Lock`

Throws:

`IllegalMonitorStateException` - if the current thread does not hold this lock

newCondition

```
public Condition newCondition()
```

Returns a `Condition` instance for use with this `Lock` instance.

The returned `Condition` instance supports the same usages as do the `Object` monitor methods (`wait`, `notify`, and `notifyAll`) when used with the built-in monitor lock.

- If this write lock is not held when any `Condition` method is called then an `IllegalMonitorStateException` is thrown. (Read locks are held independently of write locks, so are not checked or affected. However it is essentially always an error to invoke a condition waiting method when the current thread has also acquired read locks, since other threads that could unblock it will not be able to acquire the write lock.)

- When the condition waiting methods are called the write lock is released and, before they return, the write lock is reacquired and the lock hold count restored to what it was when the method was called.
- If a thread is interrupted while waiting then the wait will terminate, an `InterruptedException` will be thrown, and the thread's interrupted status will be cleared.
- Waiting threads are signalled in FIFO order.
- The ordering of lock reacquisition for threads returning from waiting methods is the same as for threads initially acquiring the lock, which is in the default case not specified, but for *fair* locks favors those threads that have been waiting the longest.

Specified by:

`newCondition` in interface `Lock`

Returns:

the Condition object

toString

```
public String toString()
```

Returns a string identifying this lock, as well as its lock state. The state, in brackets includes either the String `"Unlocked"` or the String `"Locked by"` followed by the name of the owning thread.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this lock, as well as its lock state

isHeldByCurrentThread

```
public boolean isHeldByCurrentThread()
```

Queries if this write lock is held by the current thread. Identical in effect to `ReentrantReadWriteLock.isWriteLockedByCurrentThread()`.

Returns:

`true` if the current thread holds this lock and `false` otherwise

Since:

1.6

getHoldCount

```
public int getHoldCount()
```

Queries the number of holds on this write lock by the current thread. A thread has a hold on a lock for each lock action that is not matched by an unlock action. Identical in effect to `ReentrantReadWriteLock.getWriteHoldCount()`.

Returns:

the number of holds on this lock by the current thread, or zero if this lock is not held by the current thread

Since:

1.6

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.